

Chapter 3. Image processing

Contents

3.1 Pixel Processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

3.3 Linear Image Filters

3.4 Non-Linear Image Filters

Chapter 3. Image processing

3.1 Pixel Processing

We can transform the brightness value of a pixel based on the value itself, independently of its location or the values of other pixels in the image.

It is basically a mapping of one brightness value to another brightness value or one color to another color.

$f(x,y)$ is the value, $g(x,y)$ is the result of the transformation T .

$$g(x, y) = T(f(x, y))$$

Chapter 3. Image processing

3.1 Pixel Processing

The following figures illustrate an example of such transformations.



Original (f)



f-128



f+128



255-f

Chapter 3. Image processing

3.1 Pixel Processing



Original (f)



$f/2$



$f*2$



$0.3*f_R+0.6*f_G+0.1*f_B$

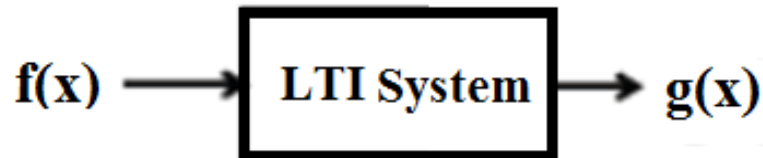
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

We will present this concept using one-dimensional signals before extending to multiple dimensions.

Figure below shows an LTI system with input $f(x)$ and output $g(x)$.



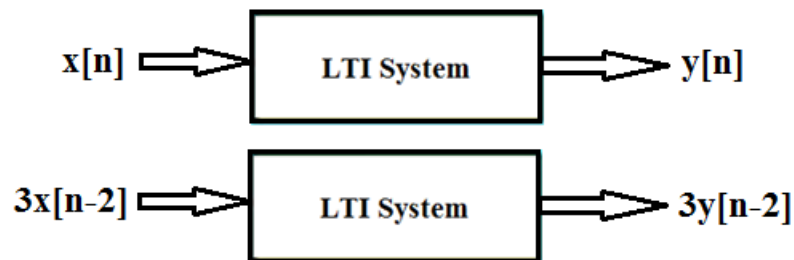
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

Linear: The output is scaled by the same amount as the input. Its response to a weighted sum of inputs is equal to the weighted sum of its responses to each of the inputs.

Time invariant: The output is delayed the same as the input. If delaying the input to the system delays the output by the same amount of time.



Chapter 3. Image processing

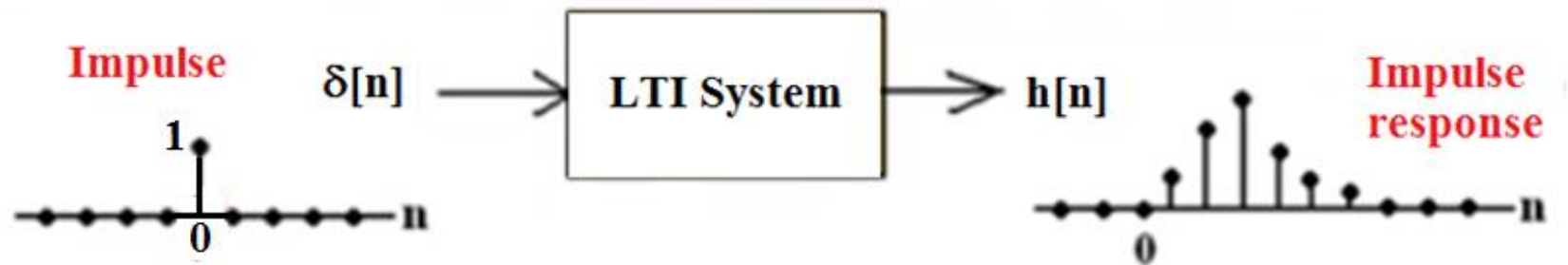
3.2 LTI Systems (Linear Time Invariant) and Convolution

The unit impulse response for DT systems

The simplest input signal to a system is a single unit impulse.

$H[n]$ = System output (response) to a unit impulse input $\delta[n]$.

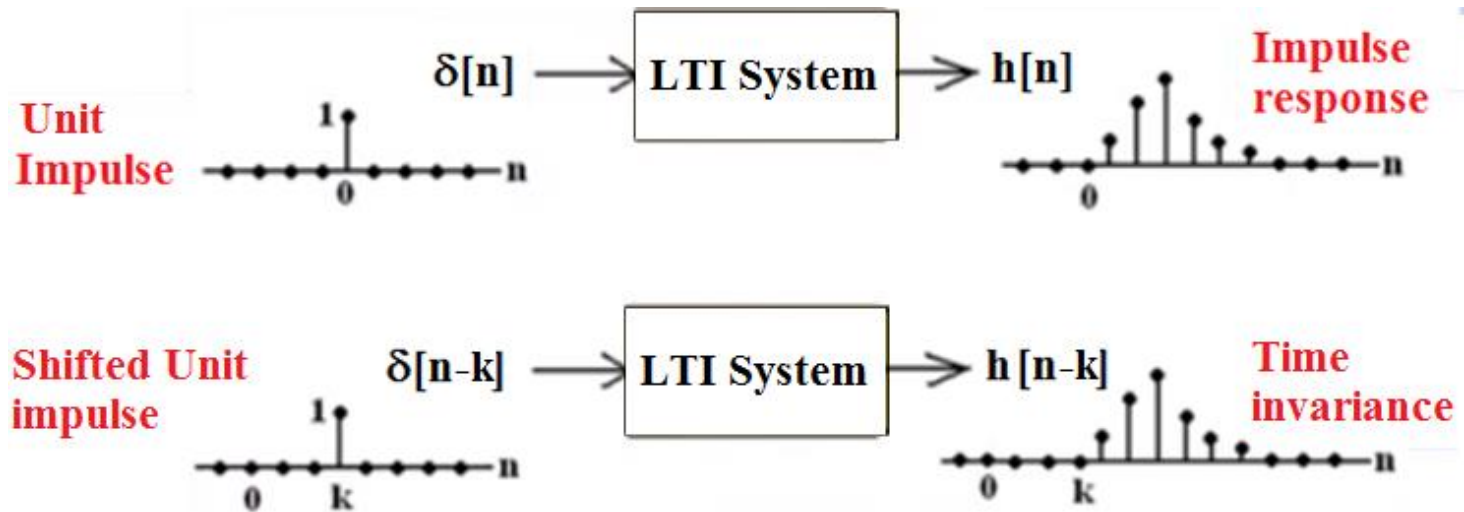
The importance of the unit impulse response $h[n]$ is that for LTI Systems (Linear Time Invariant) we can use superposition to construct the output $y[n]$ for any input $x[n]$ just by scaling and shifting $h[n]$. Thus, the unit impulse response completely characterizes the system.



Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

DT systems response for shifted/scaled unit impulse



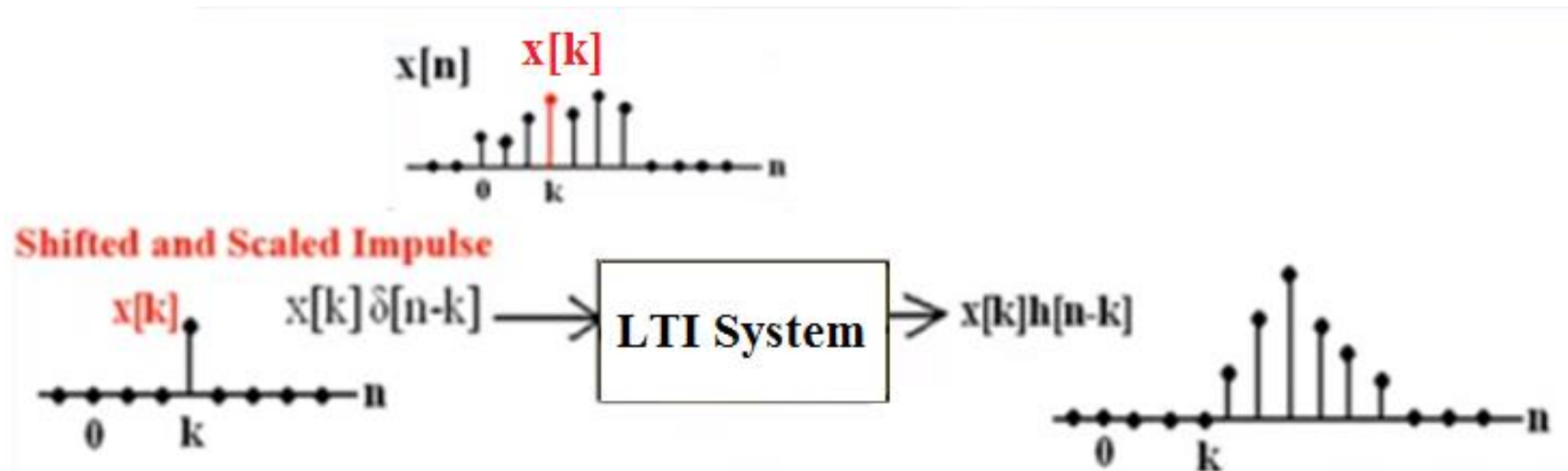
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

DT systems response for shifted/scaled unit impulse.

We suppose that we have some sequences $x[n]$ with the value $x[k]$ at $n = k$.

Output is now scaled by the value of $x[k]$.



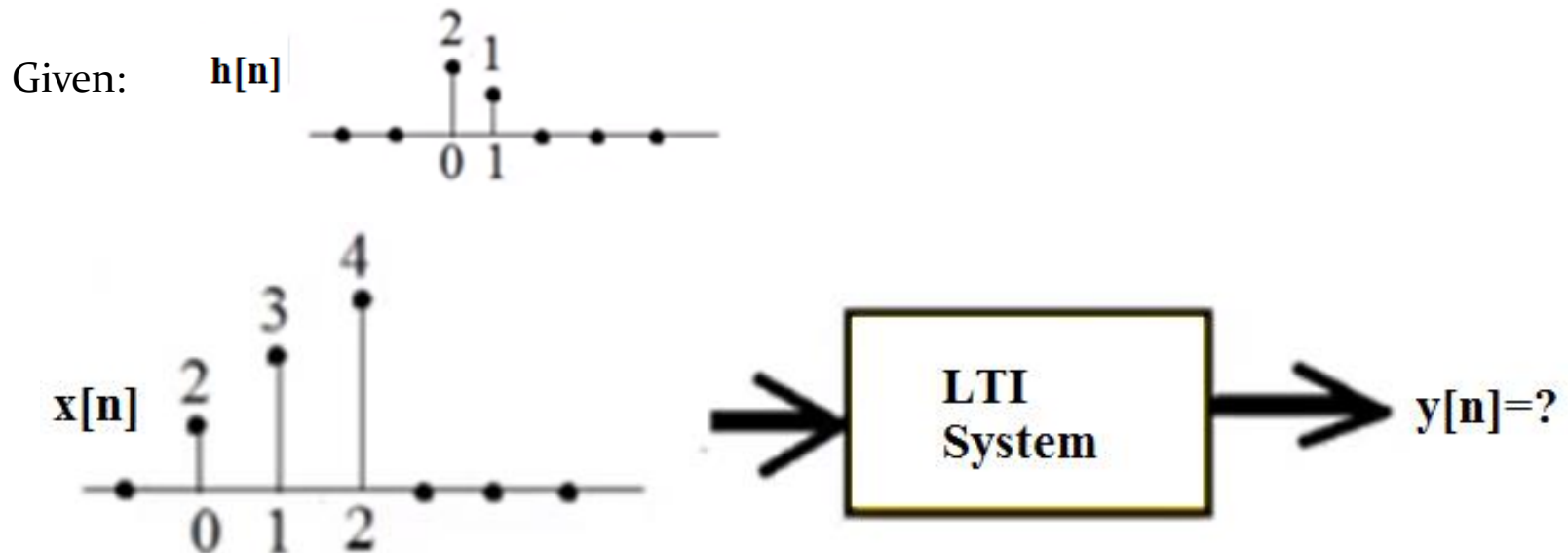
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Example:

Given the following impulse response $h[n]$ for the system, find the output $y[n]$ for the input $x[n]$ shown.

Hint: Treat each member of $x[n]$ individually as scaled/shifted impulses. Determine the output for each member of $x[n]$ separately as scaled/shifted versions of $h[n]$. Add all results to get $y[n]$.

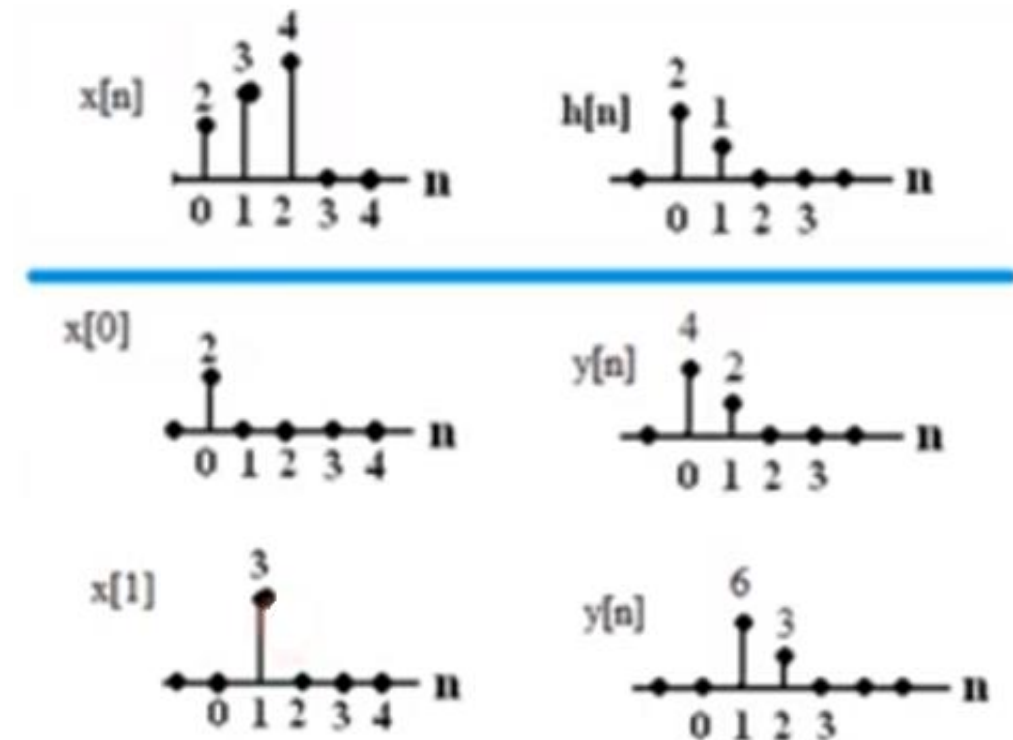


Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Solution:

The procedure is called convolution or convolution sum.

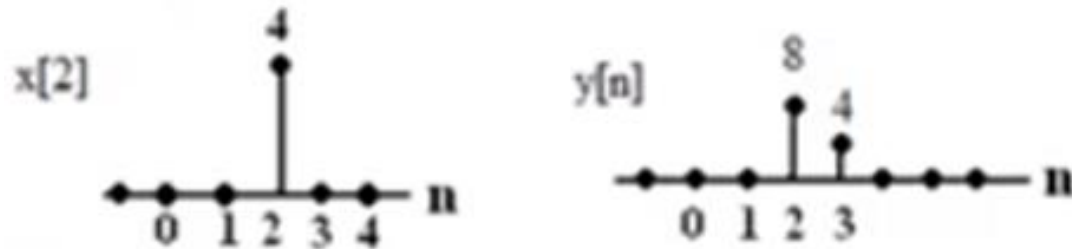


Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

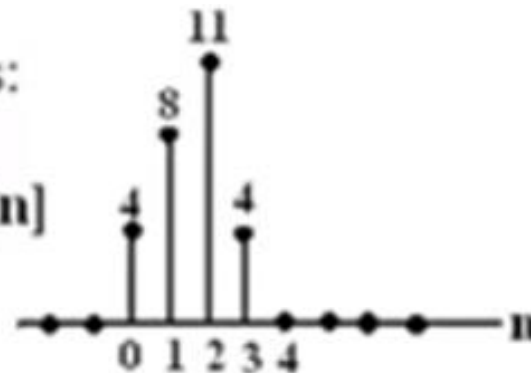
Solution:

The procedure is called convolution or convolution sum.



Add results:

$$y[n] = x[n] * h[n]$$



Chapter 3. Image processing

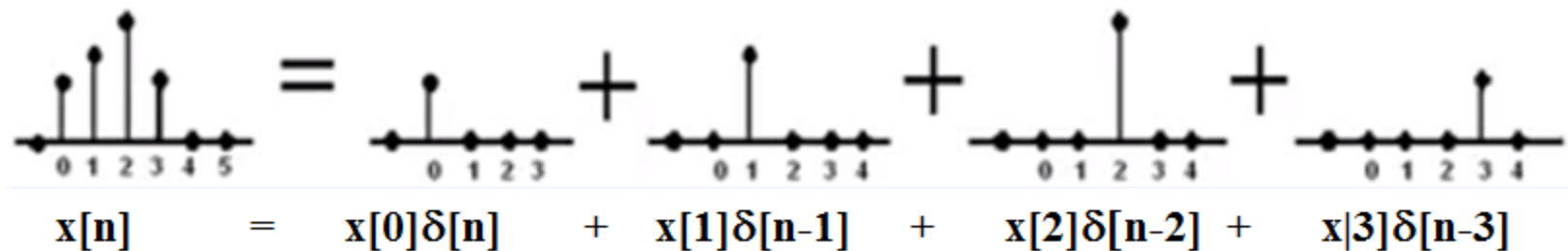
3.2 LTI Systems (Linear Time Invariant) and Convolution

Review DT Sampling (shifting) property

$$x[k] = \sum_{n=-\infty}^{\infty} x[n]\delta[k-n] \quad x[n] = \sum_{k=-\infty}^{\infty} x[k]\delta[n-k]$$

Interpretations:

1. The δ -function samples the n -th term of the sequence $x[k]$
2. The sequence $x[n]$ is represented as a weighted sum of shifted unit impulses

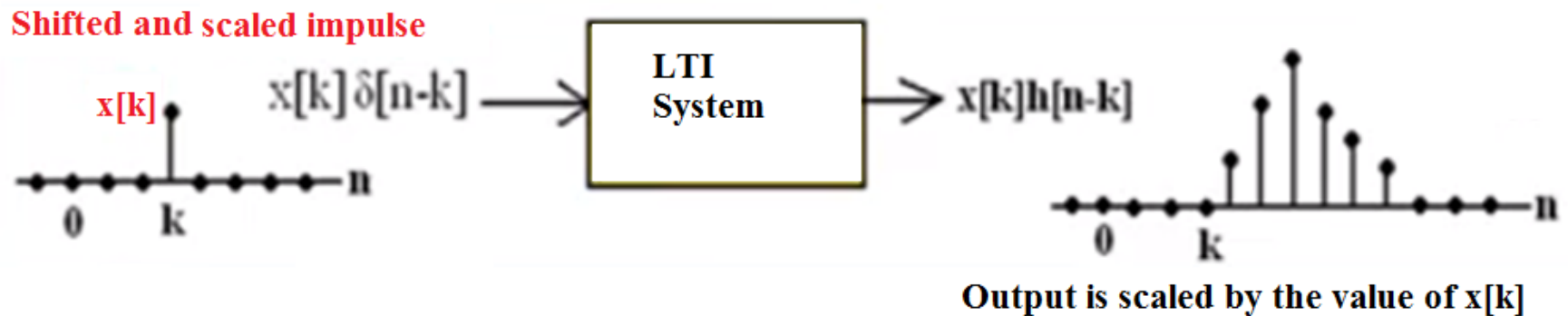

$$x[n] = x[0]\delta[n] + x[1]\delta[n-1] + x[2]\delta[n-2] + x[3]\delta[n-3]$$

Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

DT systems response for shifted/scaled unit impulse



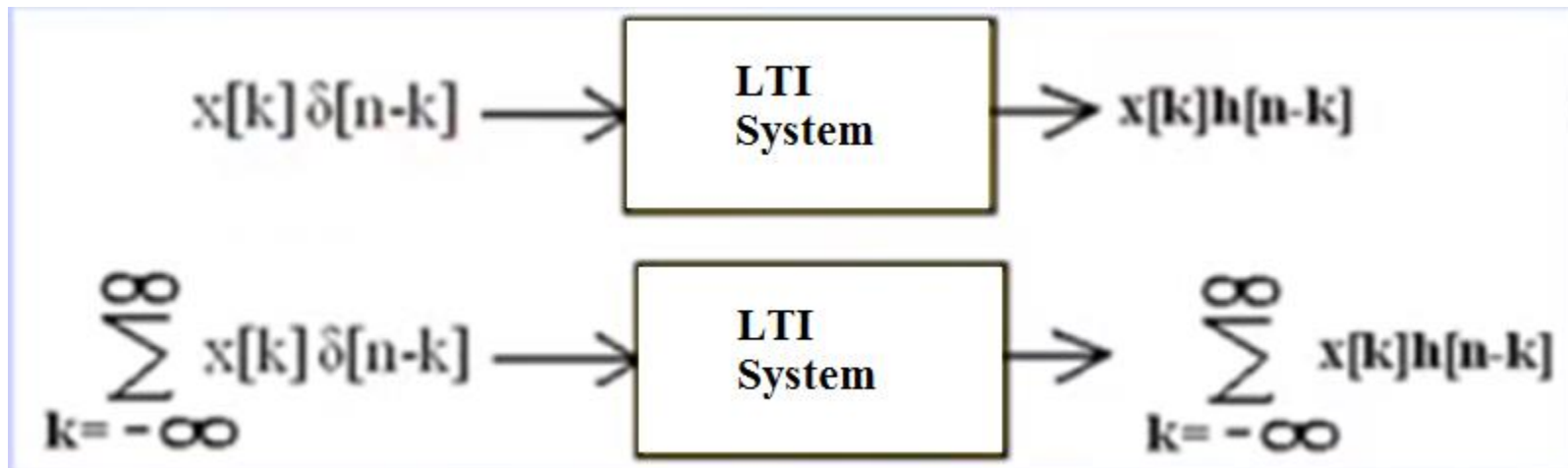
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

DT system output for general input sequence

Next sum the previous result over all values of k . Since system is linear, the output is also summed (superposition principle).



Chapter 3. Image processing

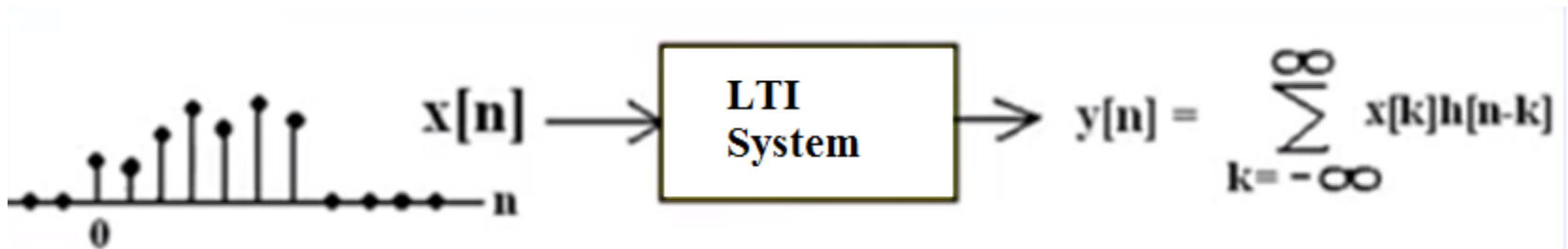
3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

DT system output for general input sequence

Next sum the previous result over all values of k . Since system is linear, the output is also summed (superposition principle).

$$x[n] = \sum_{k=-\infty}^{\infty} x[k] \delta[n - k]$$



Chapter 3. Image processing

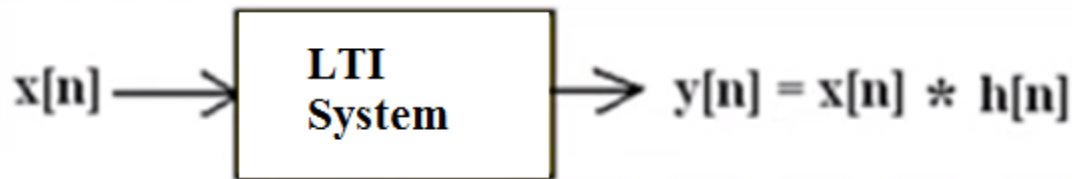
3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

The system's output $y[n]$ for any input sequence $x[n]$ just by knowing the impulse response $h[n]$.

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

$$y[n] = x[n] * h[n]$$



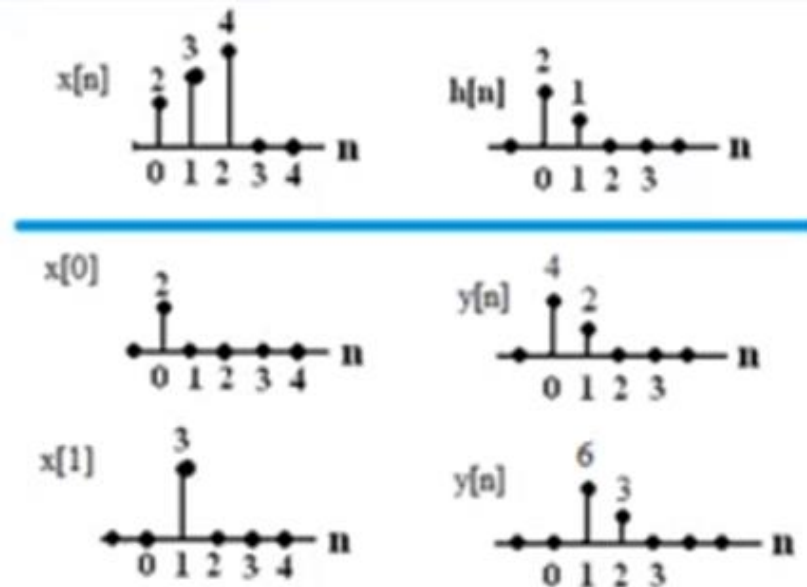
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

Given $x[n]$ and $h[n]$, find the output $y[n]$.

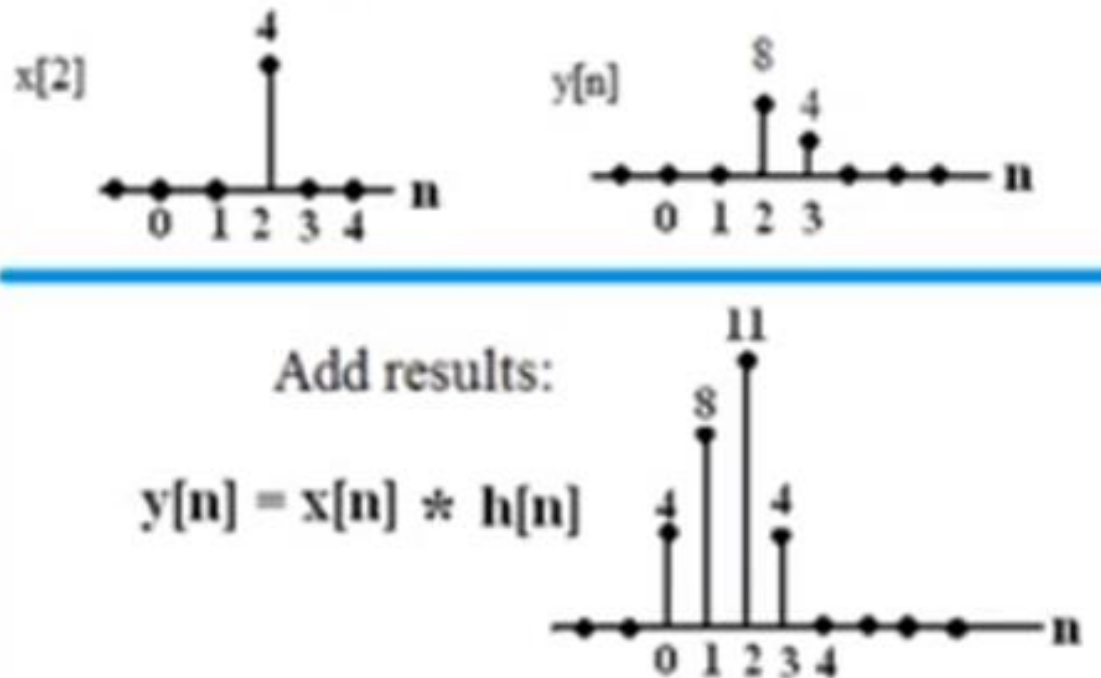
1. Break up the input $x[n]$ into individual shifted/scaled impulses
2. Determine the output $y[n]$ for each of these impulses by shifting/scaling $h[n]$ accordingly.
3. Add all the results together to get $y[n]$.



Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires



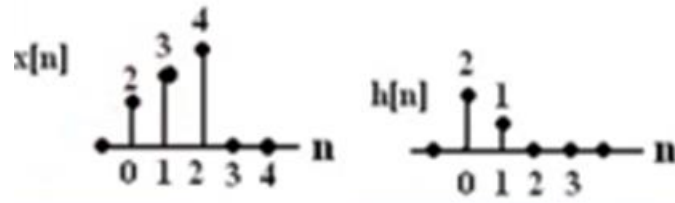
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

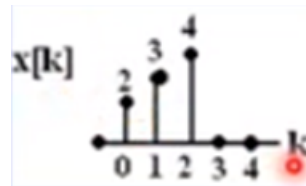
Systèmes invariant aux décalage linéaires

Given $x[n]$ and impulse response $h[n]$, find the output $y[n]$.

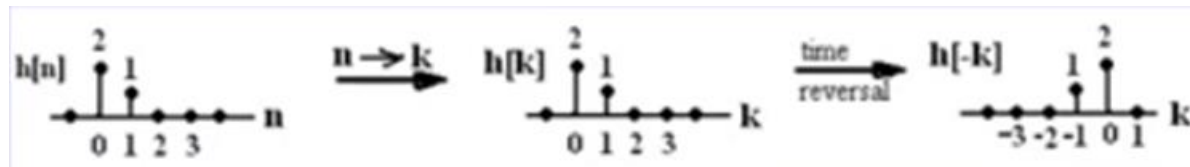
$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$



Step1: Rename $x[n] \rightarrow x[k]$



Step2: Change $h[n]$ into $h[-k]$

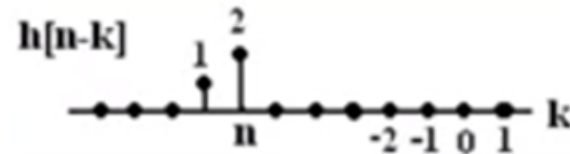


Chapter 3. Image processing

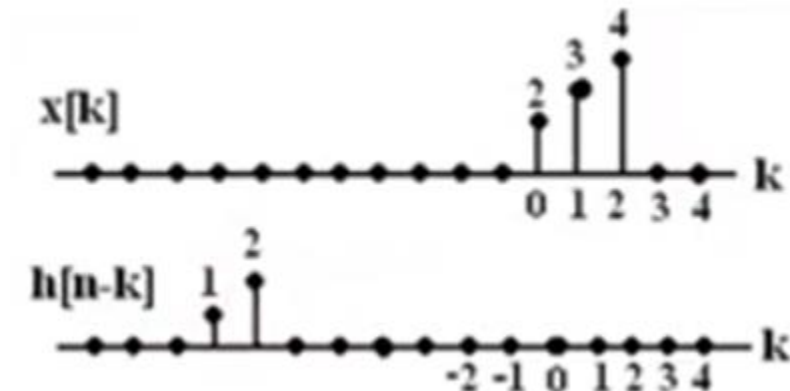
3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

Step3: Shift $h[-k]$ by n , where $n < 0$.



Step4: Note that for all $n < 0$ in this example, there is no overlap $x[k]$ and $h[n-k]$, so the product $x[k]h[n-k]=0$. Thus $y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$.



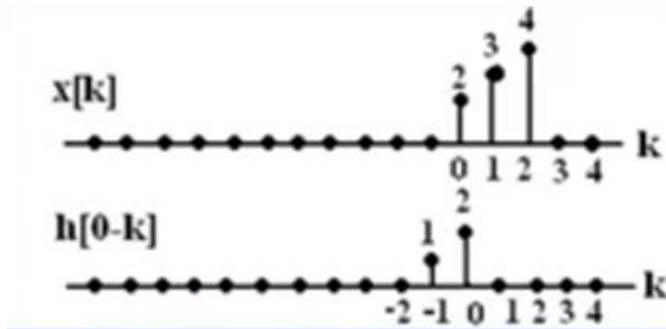
$$y[n] = 0 \text{ for } n < 0$$

Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

Step 5: Slide $h[n-k]$ to the right by increasing n . Note that when $n=0$, the sequences start to overlap and produce non-zero terms.



$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k].$$

$$y[0] = 2 \times 2 = 4$$

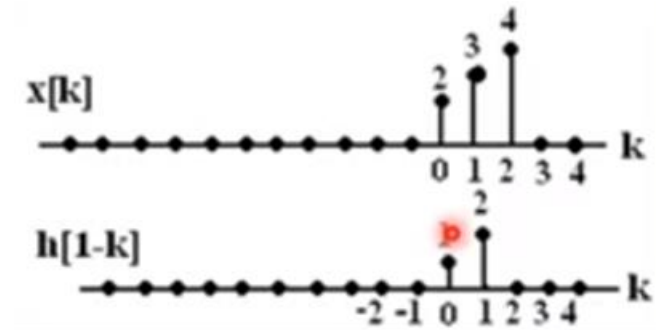
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

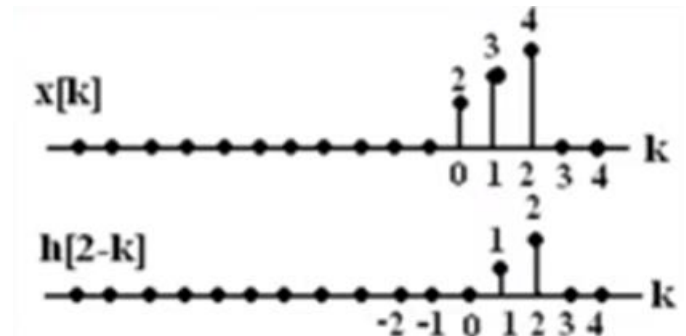
Step 6: Slide $h[n-k]$ over to the right one more step: $n=1$.

$$y[1] = 2 \times 1 + 3 \times 2 = 8$$



Step 7: Slide $h[n-k]$ over to the right one more step: $n=2$.

$$y[2] = 3 \times 1 + 4 \times 2 = 11$$



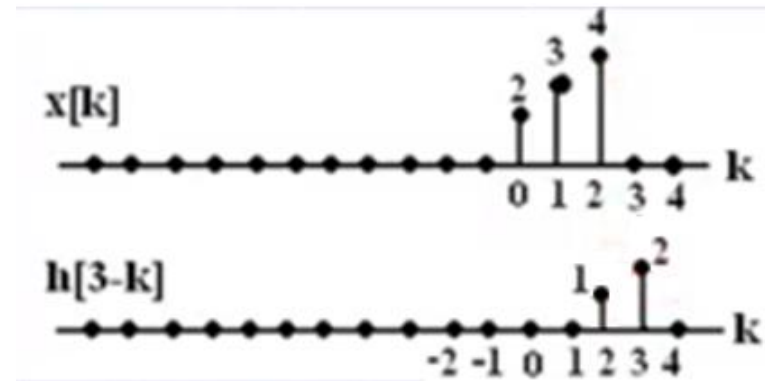
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

Step 8: Slide $h[n-k]$ over to the right one more step: $n=3$.

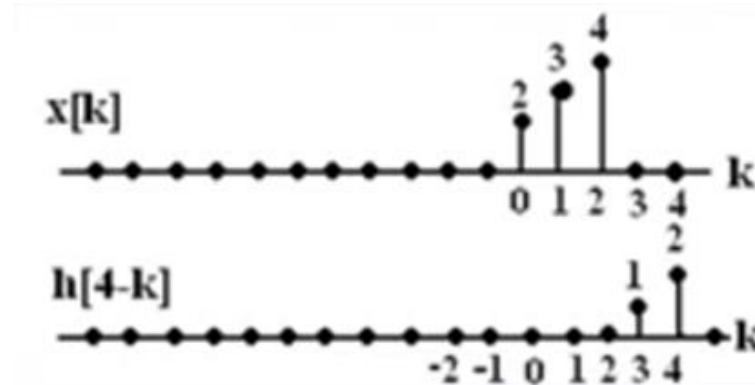
$$y[3] = 4 \times 1 = 4$$



Step 8: Slide $h[n-k]$ over to the right one more step: $n=4$.

$$y[4] = 0$$

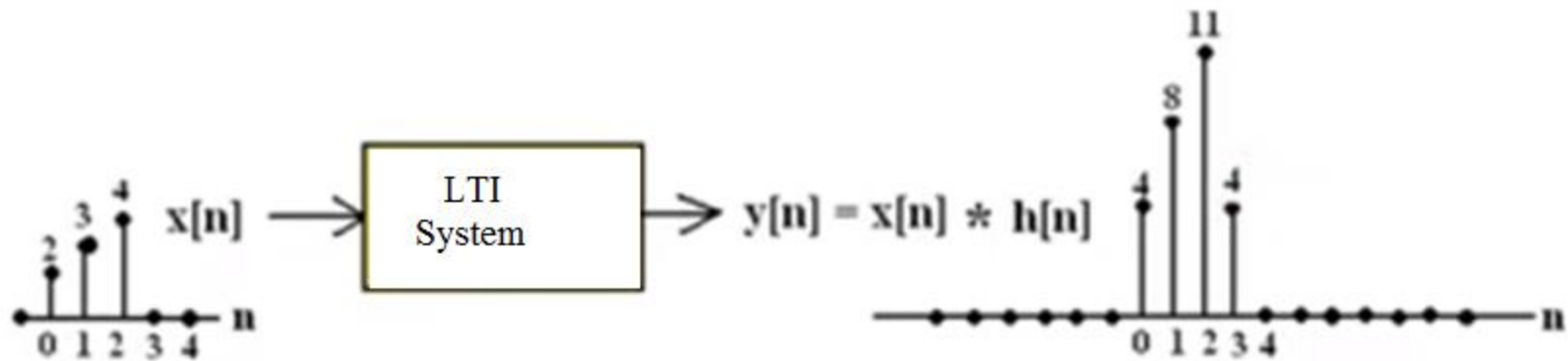
$$y[n] = 0 \text{ for } n > 3$$



Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Convolution: is a response of LTI system to an arbitrary input.



Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Systèmes invariant aux décalage linéaires

how LTI system principles apply in image convolution:

Linearity in Image Convolution: Image convolution follows the principle of linearity. This means that if you apply a filter to an image, the output can be viewed as a linear combination of the filtered responses at each pixel. This makes it easy to apply multiple filters or analyze the effects of combined filters, similar to the linearity seen in LTI systems.

Shift-Invariance in Spatial Domain: The principle of time invariance in LTI systems translates to **spatial invariance** in images. In image processing, spatial invariance means that the response of the convolution is independent of the position in the image. If you shift the image by a certain number of pixels and then apply the filter, the resulting output shifts accordingly without altering the structure of the response.

Impulse Response in 2D: In LTI systems, the impulse response fully characterizes the system's behavior. Similarly, in image convolution, the kernel (or filter) plays the role of the impulse response. By convolving the image with this kernel, we can apply various transformations. Different kernels lead to different responses, such as emphasizing edges or smoothing areas.

Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

Linearity:

If the transformations (see below) are verified, then, the system is linear.

$$f_1 \longrightarrow \boxed{\text{LSIS}} \longrightarrow g_1 \qquad f_2 \longrightarrow \boxed{\text{LSIS}} \longrightarrow g_2$$

$$\alpha f_1 + \beta f_2 \longrightarrow \boxed{\text{LSIS}} \longrightarrow \alpha g_1 + \beta g_2$$

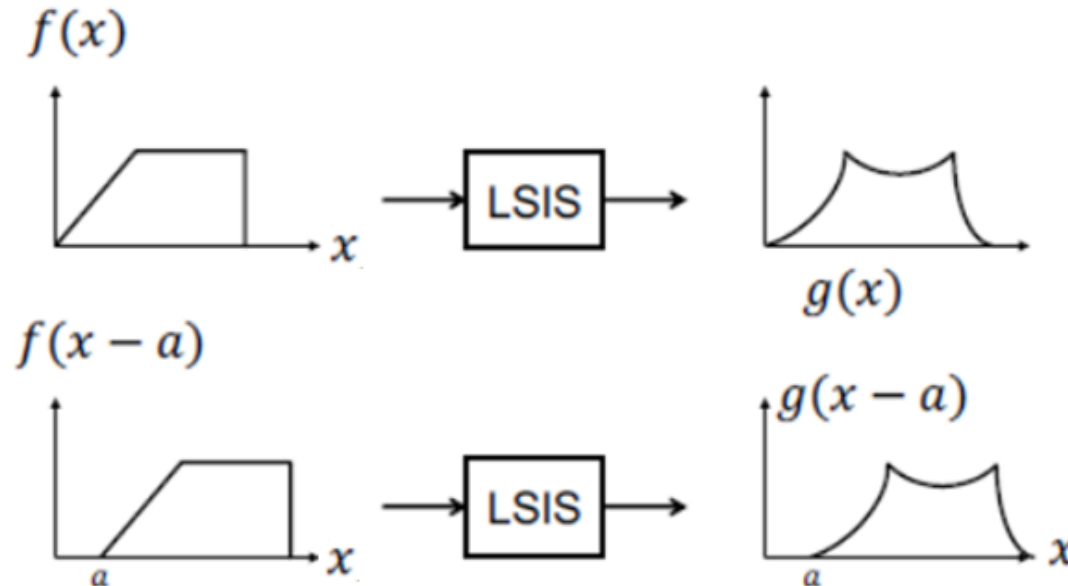
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

Shift Invariance:

Any system that satisfies linearity and shift invariance is a linear shift invariant system. The following figure illustrates that $f(x)$ is shift invariant.



Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Convolution:

Convolution is critically important for Linear Time-Invariant (LTI) systems because it provides a straightforward method for calculating the system's response to any arbitrary input, using just the system's **impulse response**.

Response Prediction with Impulse Response In an LTI system, knowing the impulse response $h(t)$ is enough to determine the response to any input signal $x(t)$. If $h(t)$ is the response to a single impulse, convolution with $x(t)$ gives the complete response $y(t)$:

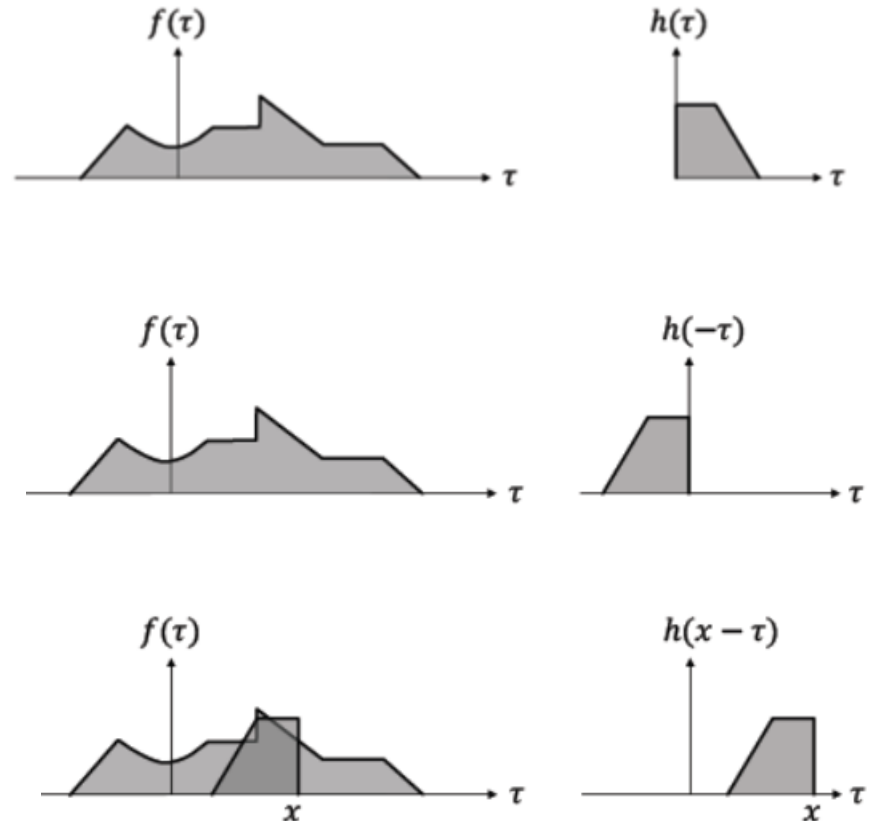
$$y(t) = x(t) * h(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau) d\tau$$

Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Convolution:

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$



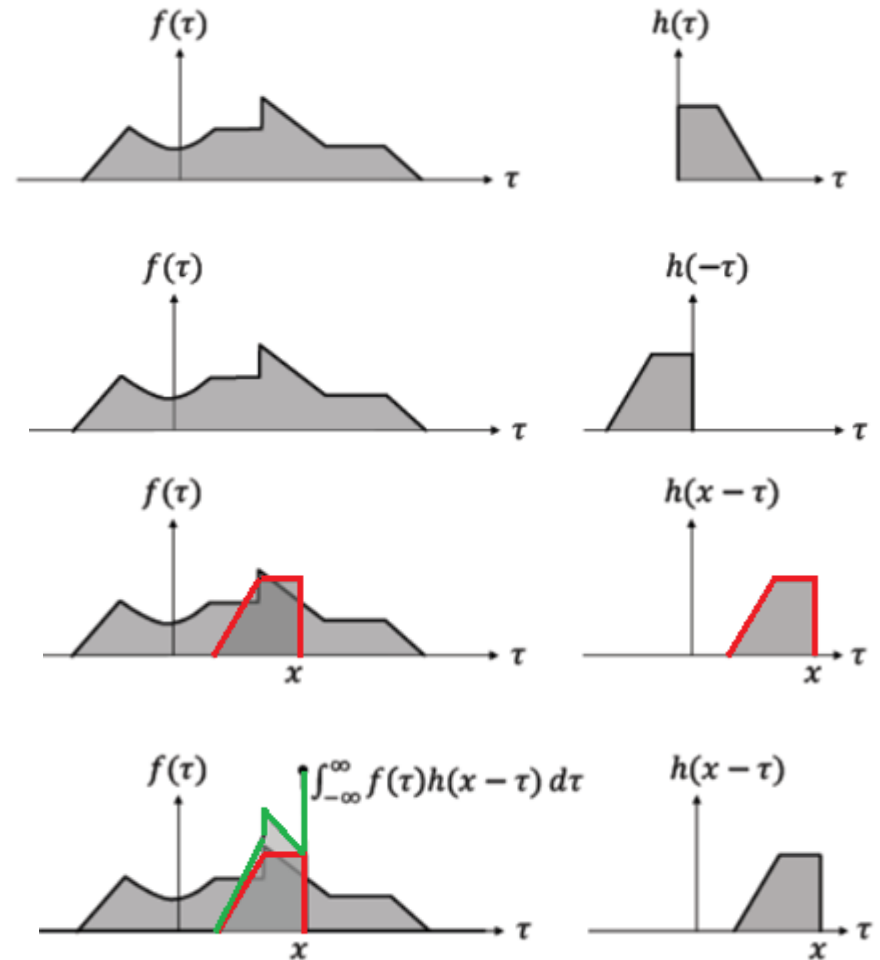
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Convolution:

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

If we take the product $f(\tau)h(x - \tau)$ of these two overlapping functions and integrate it from minus infinity to infinity, this gives us a single number, which is the result of the convolution at the point x .



Chapter 3. Image processing

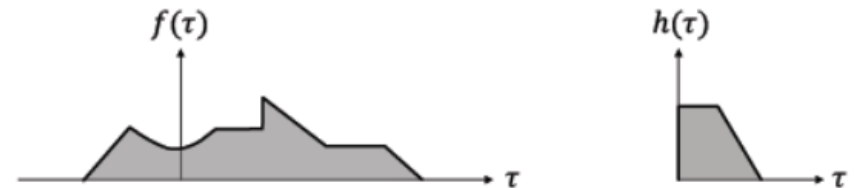
3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

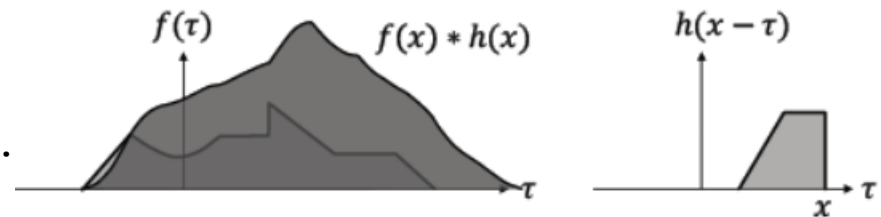
$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

Convolution:

To find the entire function $g(x)$, we would flip the function $h(\tau)$ and then move it to minus infinity, that is the shift x in $h(x - \tau)$ equals minus infinity.



We then vary the shift from minus infinity to plus infinity by sliding the function $h(-\tau)$ over $f(\tau)$ from left to right. For each shift value x we find the product of the two functions and then the integral of the product. This gives us the entire function $g(x)$, which is the result of the convolution.



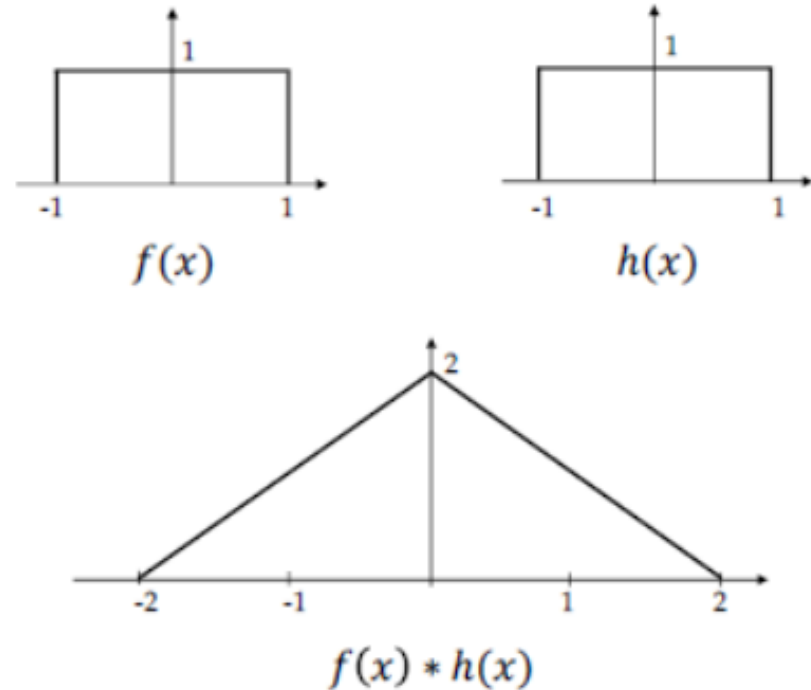
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

Convolution:

Example of convolution.



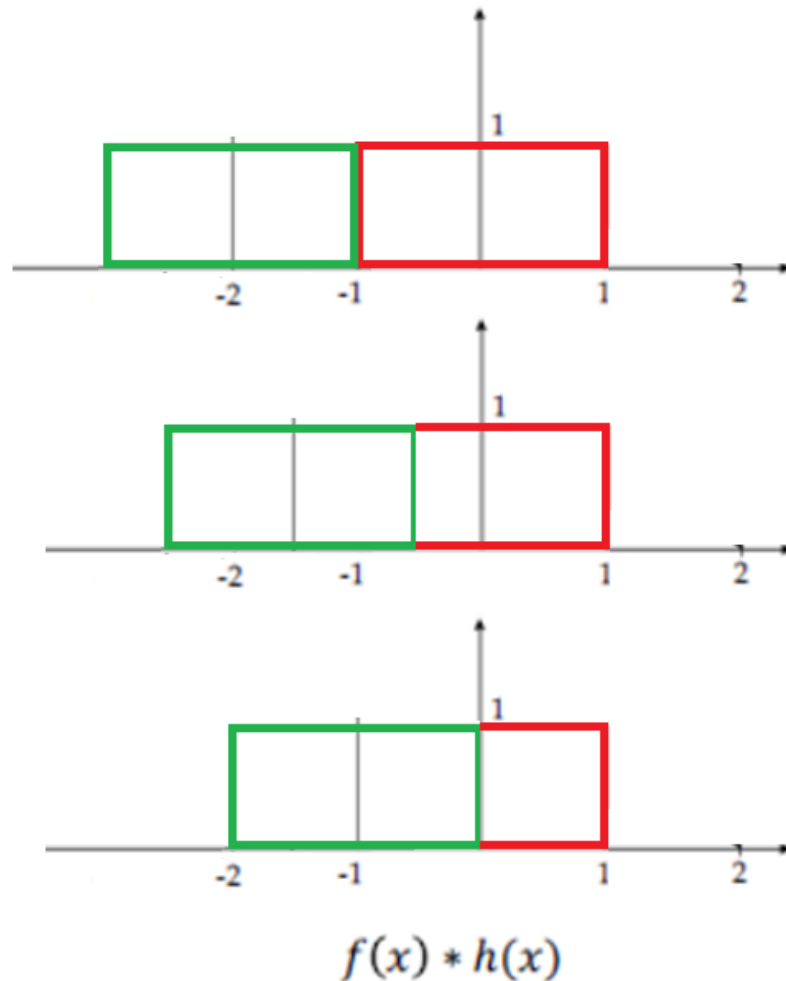
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

Convolution:

Example of convolution.



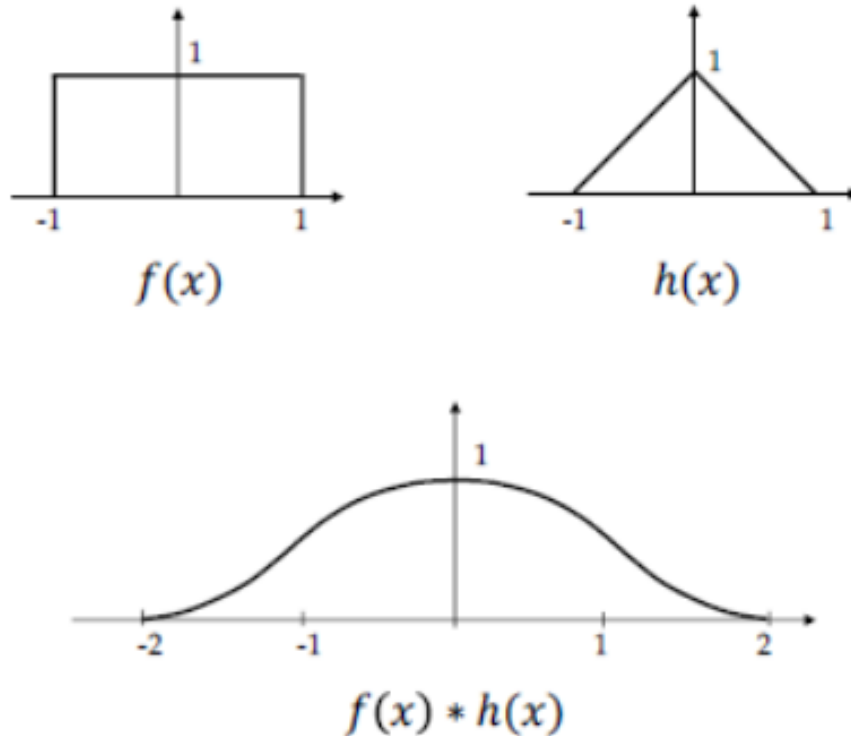
Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

Convolution:

Example of convolution.



Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

Convolution is LTI :

Linearity:

We assume that:

$$g_1(x) = \int_{-\infty}^{\infty} f_1(\tau)h(x - \tau) d\tau \quad , \quad g_2(x) = \int_{-\infty}^{\infty} f_2(\tau)h(x - \tau) d\tau$$

$$\int_{-\infty}^{\infty} (\alpha f_1(\tau) + \beta f_2(\tau))h(x - \tau) d\tau$$

$$= \alpha \int_{-\infty}^{\infty} f_1(\tau)h(x - \tau) d\tau + \beta \int_{-\infty}^{\infty} f_2(\tau)h(x - \tau) d\tau$$

$$= \alpha g_1(x) + \beta g_2(x)$$

Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of an LTI

Convolution is LTI : Shift Invariance

We assume that:

$$g(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$

We shift the function f by the value a

$$\int_{-\infty}^{\infty} f(\tau - a)h(x - \tau) d\tau = \int_{-\infty}^{\infty} f(\mu)h(x - a - \mu) d\mu = g(x - a)$$

Where: $\mu = \tau - a$

Chapter 3. Image processing

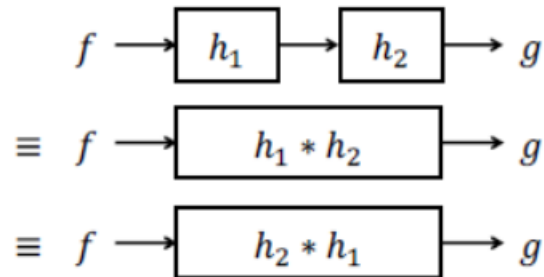
3.2 LTI Systems (Linear Time Invariant) and Convolution

Properties of convolution

Commutative $a * b = b * a$

Associative $(a * b) * c = a * (b * c)$

Cascaded System

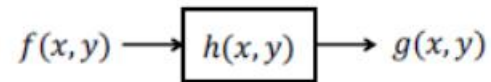


Chapter 3. Image processing

3.2 LTI Systems (Linear Time Invariant) and Convolution

Convolution 2D

LSIS:



Convolution:

$$g(x, y) = \iint_{-\infty}^{\infty} f(\tau, \mu) h(x - \tau, y - \mu) d\tau d\mu$$

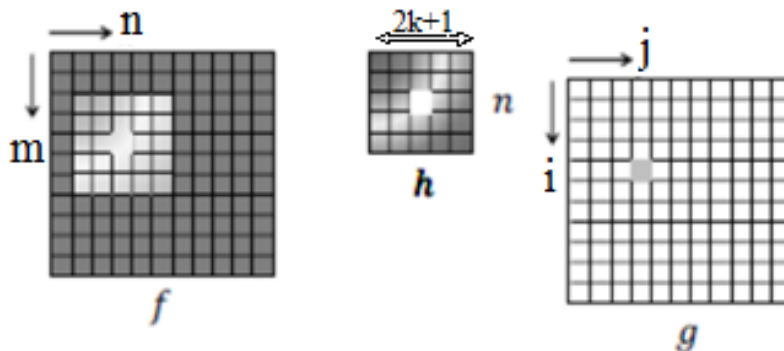
Chapter 3. Image processing

3.3 Linear Image Filters

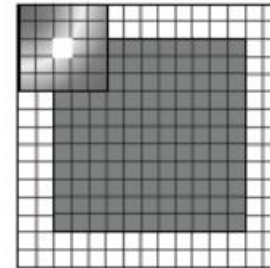
Convolution with Discrete Images

$$f[i,j] \longrightarrow h[i,j] \longrightarrow g[i,j]$$

$$g[i,j] = \sum_{m=-k}^k \sum_{n=-k}^k f(i+m,j+n)h(m,n)$$



Border Problem



Solution:

- Ignore border
- Pad with constant value
- Pad with reflection

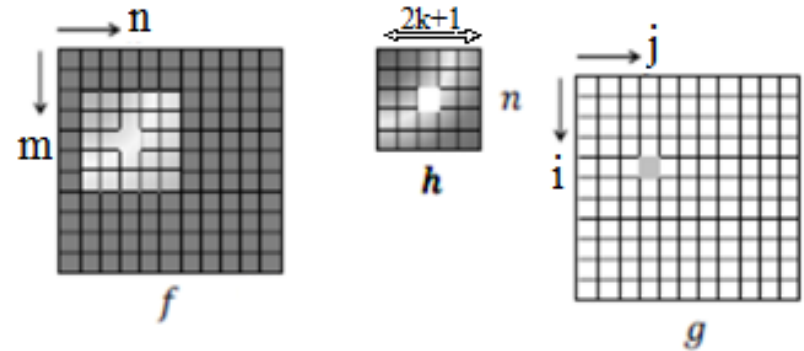
In image processing, the impulse response $h[i,j]$ is referred to as a mask, a kernel, or a filter.

Chapter 3. Image processing

3.3 Linear Image Filters

Convolution with Discrete Images

$$f[i,j] \longrightarrow h[i,j] \longrightarrow g[i,j]$$
$$g[i,j] = \sum_{m=-k}^k \sum_{n=-k}^k f(i+m, j+n) h(m,n)$$

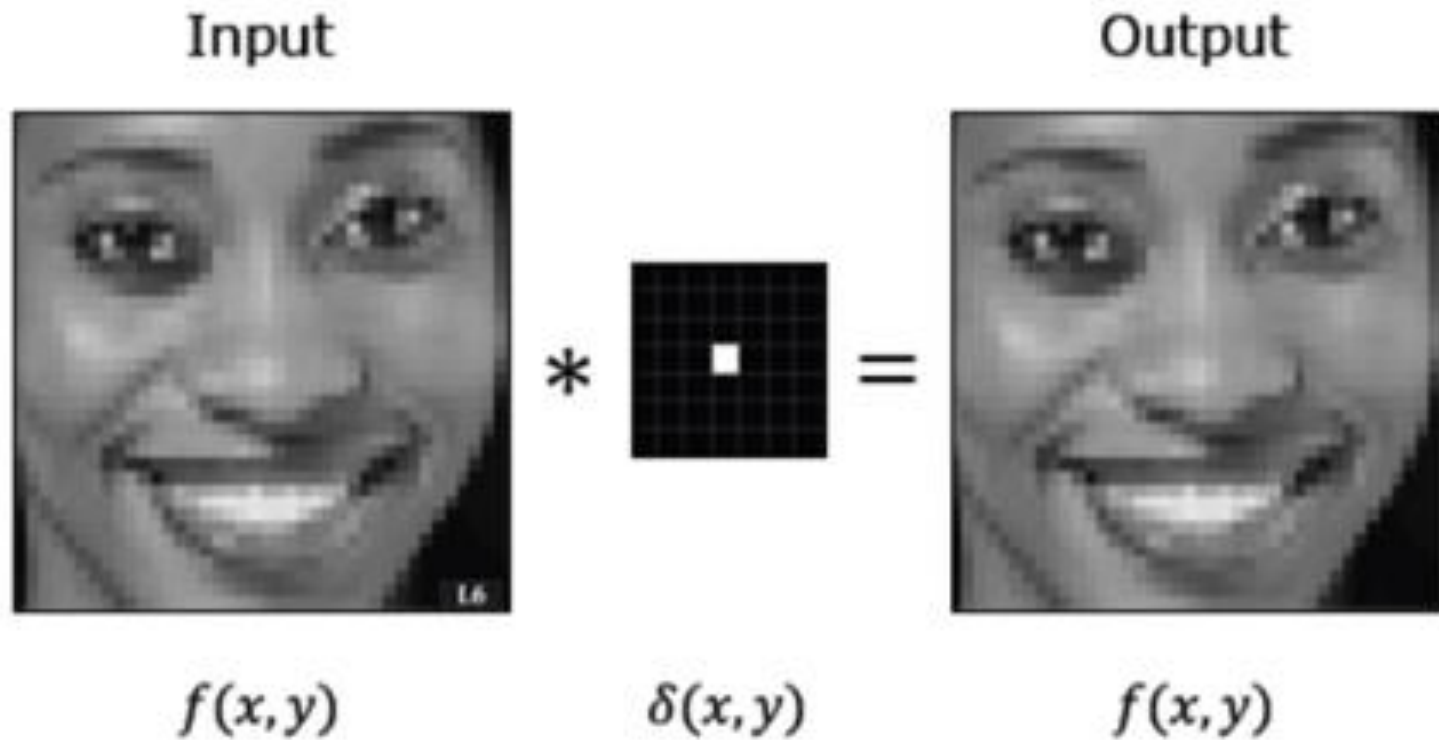


the value of the output image g at pixel location $[i,j]$ is obtained by flipping the filter h twice, overlaying it on the image f with the center of the filter at $[i,j]$, and finding the sum of the product of the pixel values of the image and the filter in the overlap region

Chapter 3. Image processing

3.3 Linear Image Filters

Convolution with Discrete Images



Chapter 3. Image processing

3.3 Linear Image Filters

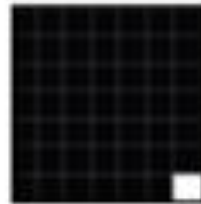
Convolution with Discrete Images

Input



$f(x, y)$

*



=

Output



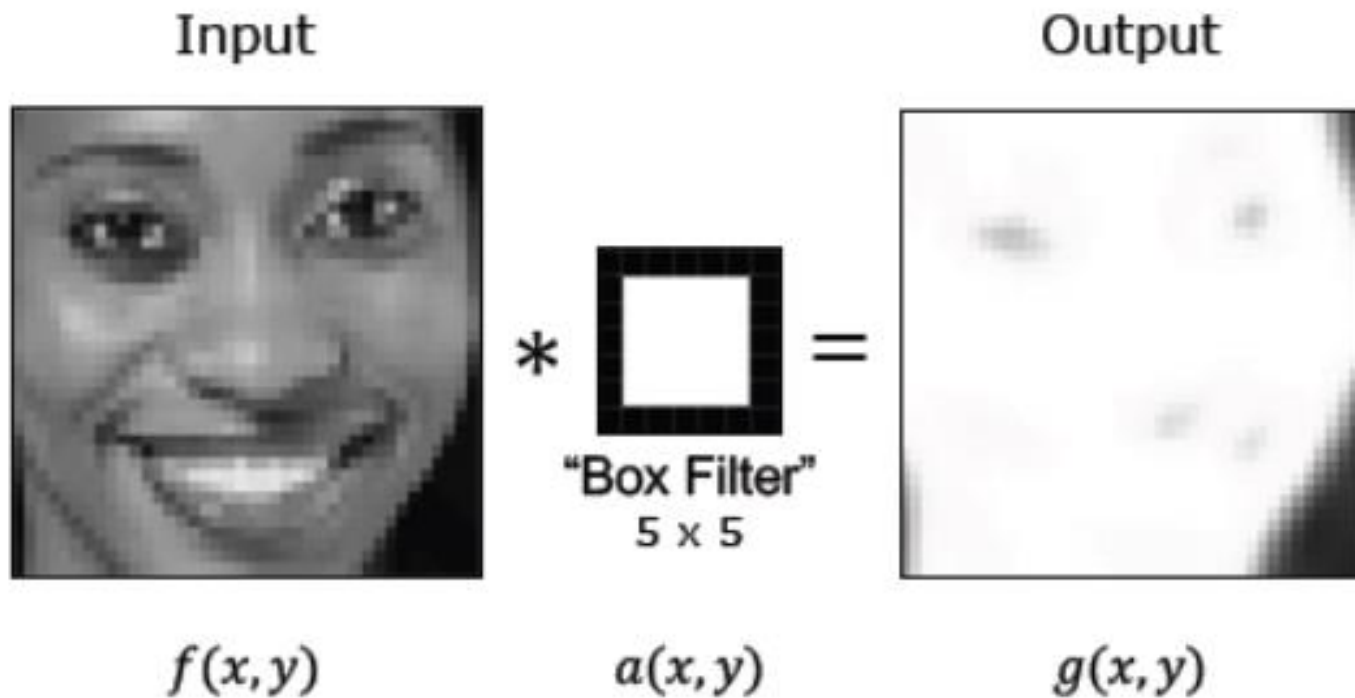
$f(x - u, y - v)$

$\delta(x - u, y - v)$

Chapter 3. Image processing

3.3 Linear Image Filters

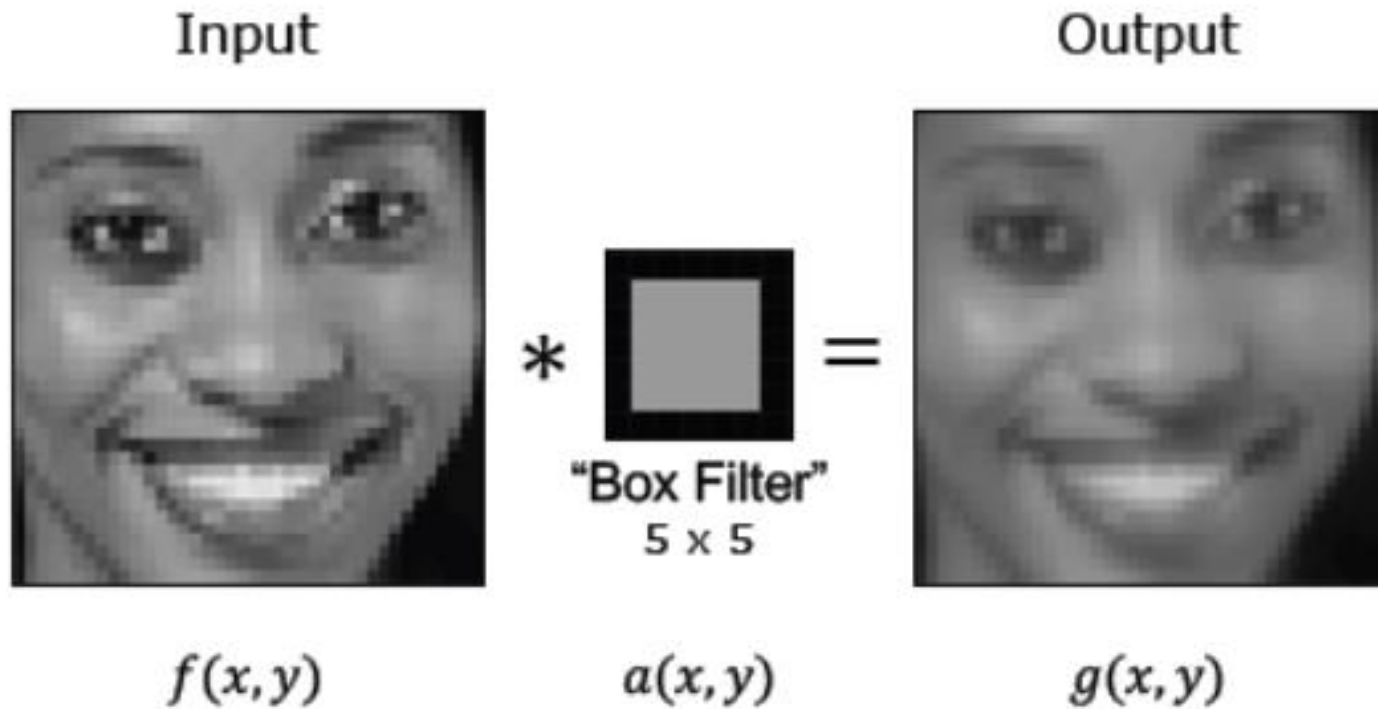
Convolution with Discrete Images



Chapter 3. Image processing

3.3 Linear Image Filters

Convolution with Discrete Images



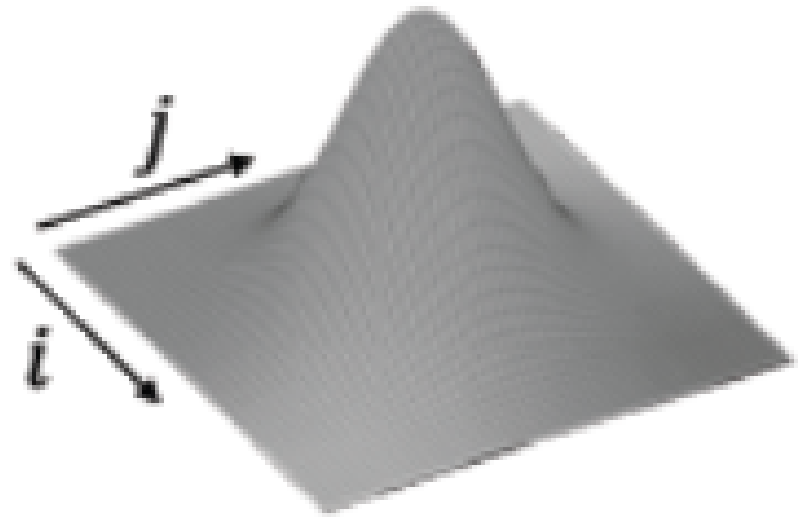
Chapter 3. Image processing

3.3 Linear Image Filters

Gaussian Kernel: A Fuzzy Filter

$$n_{\sigma}[i, j] = \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2}\left(\frac{i^2+j^2}{\sigma^2}\right)}$$

σ^2 : Variance



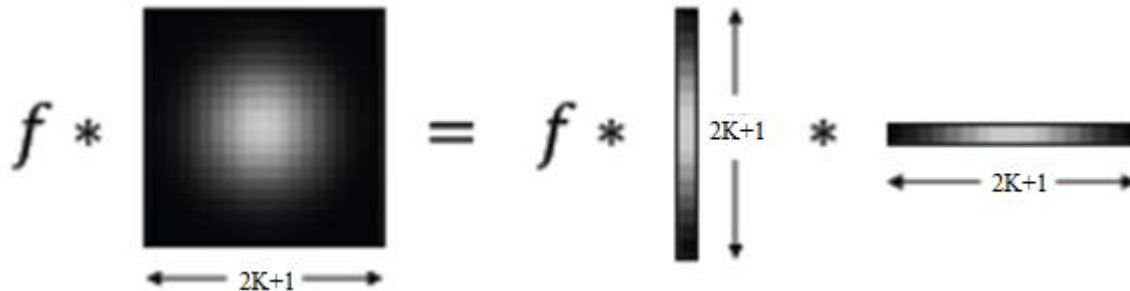
Chapter 3. Image processing

3.3 Linear Image Filters

Gaussian Kernel: is separable.

$$g(i,j) = \frac{1}{2\pi\sigma^2} \sum_{m=-k}^k \sum_{n=-k}^k f(i+m, j+n) e^{\frac{-(m^2+n^2)}{2\sigma^2}}$$

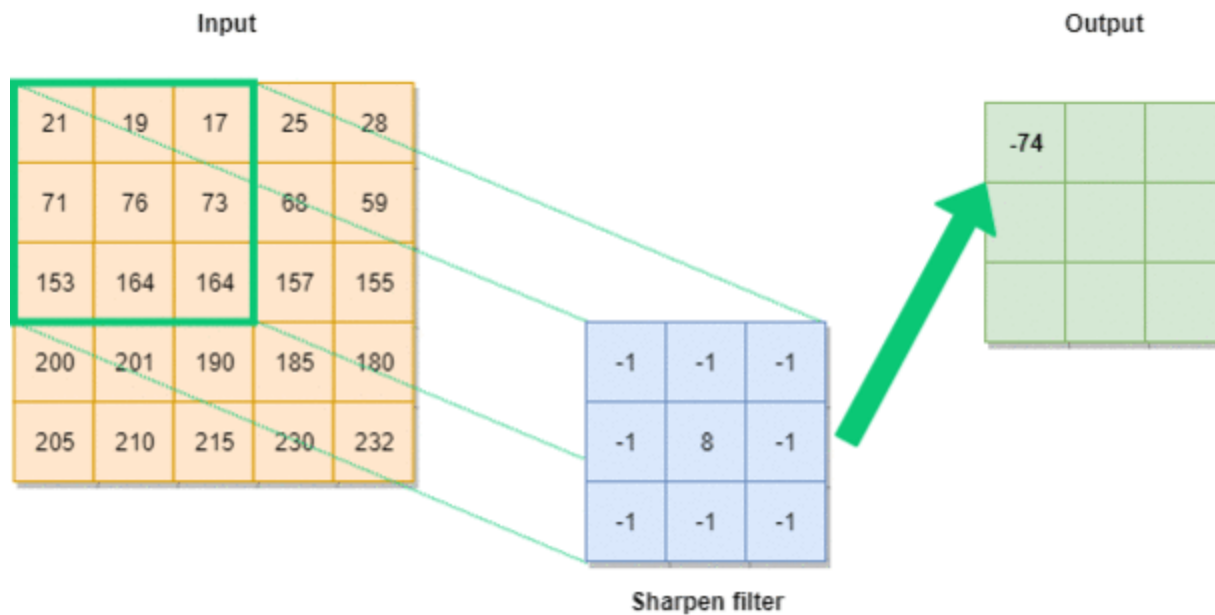
$$g(i,j) = \frac{1}{2\pi\sigma^2} \sum_{m=-k}^k e^{\frac{-(m^2)}{2\sigma^2}} \sum_{n=-k}^k e^{\frac{-(n^2)}{2\sigma^2}} f(i+m, j+n)$$



Chapter 3. Image processing

3.3 Linear Image Filters

Convolution: How is computed (Animated image (gif)):



Chapter 3. Image processing

3.3 Linear Image Filters

Convolution: Computation of the kernel and convolution using Python

```
import math
import cv2
import numpy as np
import matplotlib.pyplot as plt

def gkernel(l=5, sig=5):
    "Gaussian Kernel Creator via given length and sigma"
    ax = np.linspace(-(l - 1) / 2., (l - 1) / 2., l)
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-0.5 * (np.square(xx) +
                           np.square(yy)) / np.square(sig))
    return kernel / np.sum(kernel)
```

Chapter 3. Image processing

3.3 Linear Image Filters

Convolution: Computation of the kernel and convolution using Python

```
img = cv2.imread('/content/sample_data/image.png') # Reading Image
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # Converting Image to
        grayscale
g_kernel = gkernel(5,5) # Create gaussian kernel with 3x3(odd) size and sigma
        equals to 2
print("Gaussian Filter: ",g_kernel) # show the kernel array
dst = cv2.filter2D(gray,-1,g_kernel) #convolve kernel with image
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # convert BGR(opencv
        format) to RGB format
```

Chapter 3. Image processing

3.3 Linear Image Filters

Convolution: Computation of the kernel and convolution using Python

Sigma=5, size=5,

Gaussian Filter: $\begin{bmatrix} 0.03688345 & 0.03916419 & 0.03995536 & 0.03916419 & 0.03688345 \\ 0.03916419 & 0.04158597 & 0.04242606 & 0.04158597 & 0.03916419 \\ 0.03995536 & 0.04242606 & 0.04328312 & 0.04242606 & 0.03995536 \\ 0.03916419 & 0.04158597 & 0.04242606 & 0.04158597 & 0.03916419 \\ 0.03688345 & 0.03916419 & 0.03995536 & 0.03916419 & 0.03688345 \end{bmatrix}$

Chapter 3. Image processing

3.3 Linear Image Filters

Convolution: Computation of the kernel and convolution using Python

```
plt.figure(figsize=(18, 18))  
plt.subplot(131),plt.imshow(img),plt.title('Original Image') # visualize and  
give title  
plt.subplot(132),plt.imshow(gray,cmap="gray"),plt.title('GrayScaled Image')  
plt.subplot(133),plt.imshow(dst,cmap="gray"),plt.title('Smoothed Image  
with Gaussian Filter (sigma=2,3x3 Kernel)')  
plt.show()
```

Chapter 3. Image processing

3.3 Linear Image Filters

Convolution: Computation of the kernel and convolution using Python
Obtained result



Chapter 3. Image processing

3.3 Linear Image Filters

Convolution: Some Kernels

$$h = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Moyenneur

$$h = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Binomial-gaussien

$$h = \begin{bmatrix} 1 & -3 & 1 \\ -3 & 9 & -3 \\ 1 & -3 & 1 \end{bmatrix}$$

Rehausseur de contraste

$$h = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Sobel horizontal

$$h = \begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Gradient oblique

$$h = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

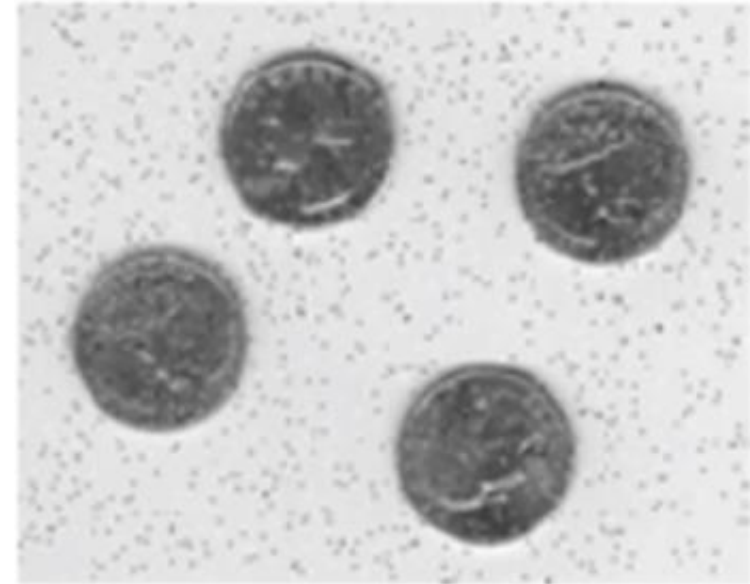
Laplacien

Chapter 3. Image processing

3.4 Non-Linear Image Filters

Smoothing an image to remove noise. The image in figure has some salt and pepper noise in it. If we simply apply a fuzzy filter such as a Gaussian, we can see that the noise is slightly diminished.

However, what we are really doing is smearing the noise out and are not really removing it. At the same time, we are also blurring out of the edges of the coins and losing some of the details within the coins.

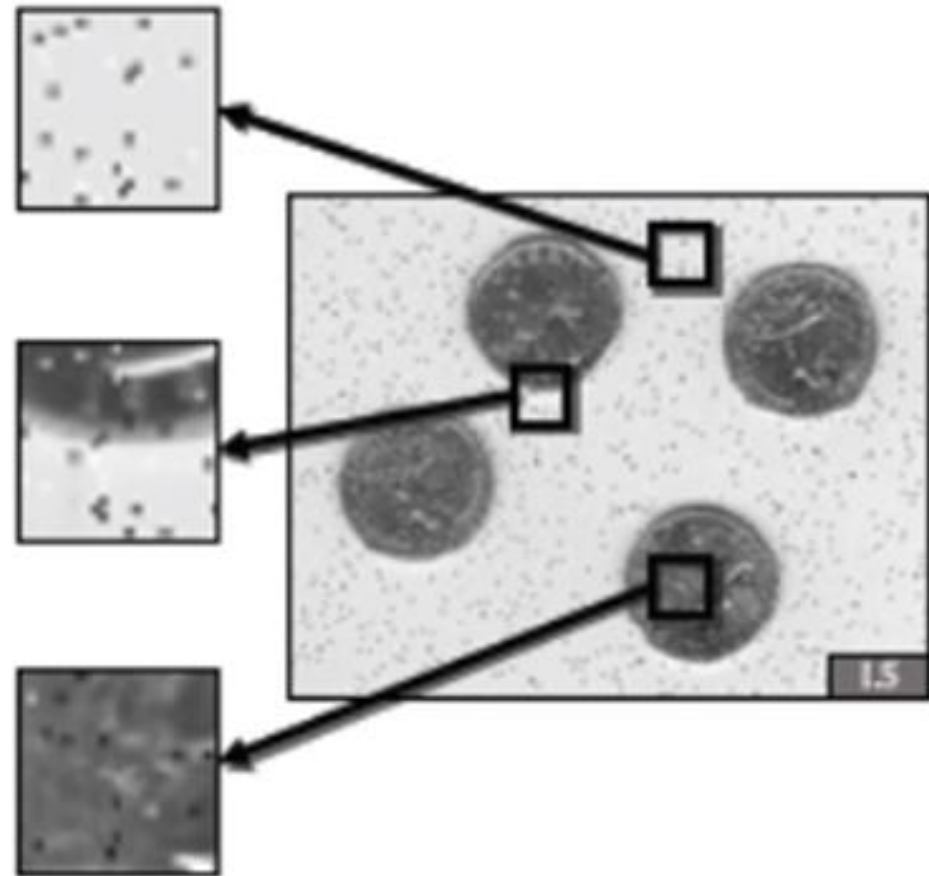


Chapter 3. Image processing

3.4 Non-Linear Image Filters

Smoothing an image to remove noise. The image in figure has some salt and pepper noise in it. If we simply apply a fuzzy filter such as a Gaussian, we can see that the noise is slightly diminished.

However, what we are really doing is smearing the noise out and are not really removing it. At the same time, we are also blurring out of the edges of the coins and losing some of the details within the coins.



Chapter 3. Image processing

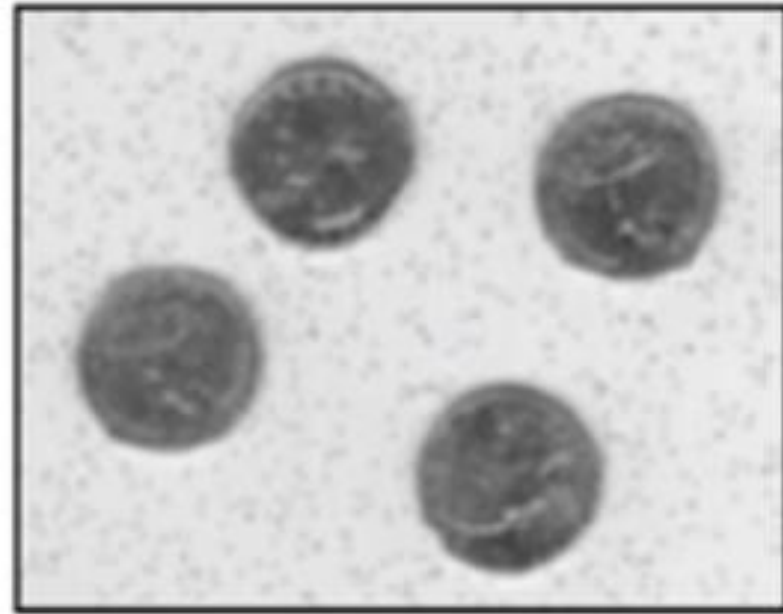
3.4 Non-Linear Image Filters

Smoothing an image to remove noise.

The image in figure has some noise. If we simply apply a fuzzy filter such as a Gaussian, we can see that the noise is slightly diminished.

However, what we are really doing is smearing the noise out and are not really removing it.

At the same time, we are also blurring out of the edges of the coins and losing some of the details within the coins.



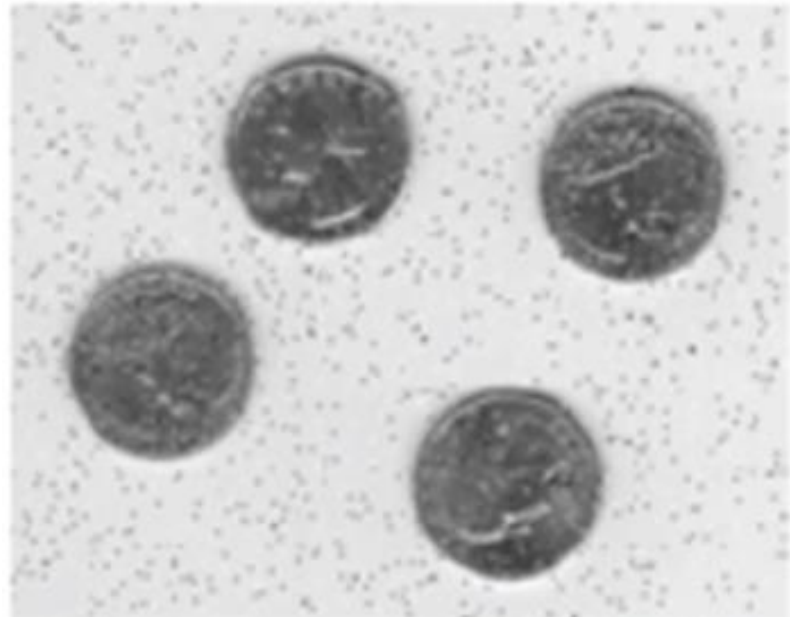
Chapter 3. Image processing

3.4 Non-Linear Image Filters

Median Filter

For noise removal, we take a different approach called **median filtering**.

For each pixel, we take all the intensity values (including its own value) within a $K \times K$ window centered at the pixel and sort them.



Chapter 3. Image processing

3.4 Non-Linear Image Filters

We then find the middle value of the sorted list, which is the median of all the intensity values. We simply use the median as the filtered output for the pixel.

When we apply median filtering with even a small filter, say $K=3$, we see that the result is a significant improvement over the original image.

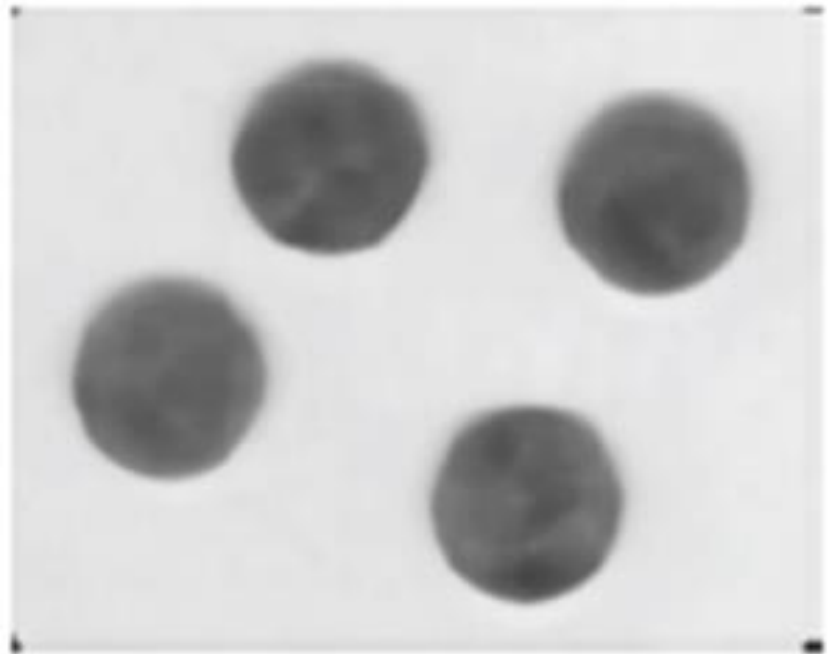


Chapter 3. Image processing

3.4 Non-Linear Image Filters

If we use a bigger median filter (example: 11×11), the noise is reduced, but the coins are also lost.

If we go to an even bigger filter, we do better in terms of the noise reduction, but even more of the details are gone.

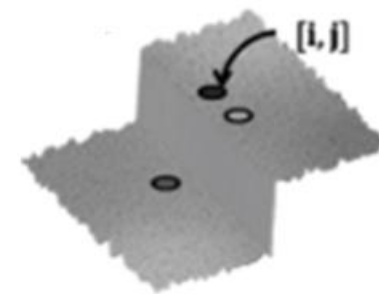
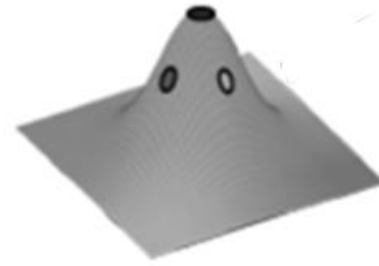


Chapter 3. Image processing

3.4 Non-Linear Image Filters

Bilateral Filter

We can fix this problem by adding another term called the brightness Gaussian, which takes the difference between the brightness of the center pixel and its neighbor. If the difference is small, it will have a large value, while if it is large, it will have a low value.

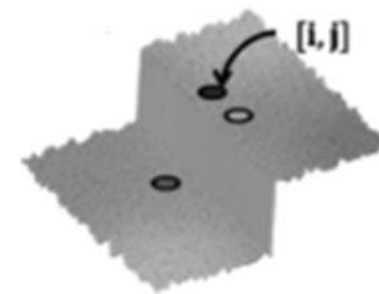
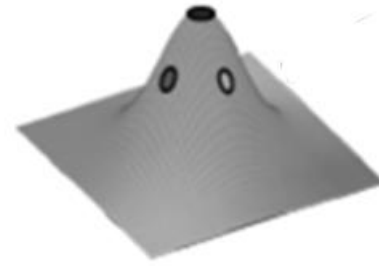


Chapter 3. Image processing

3.4 Non-Linear Image Filters

Bilateral Filter

We can fix this problem by adding another term called the brightness Gaussian, which takes the difference between the brightness of the center pixel and its neighbor. If the difference is small, it will have a large value, while if it is large, it will have a low value.



Chapter 3. Image processing

3.4 Non-Linear Image Filters

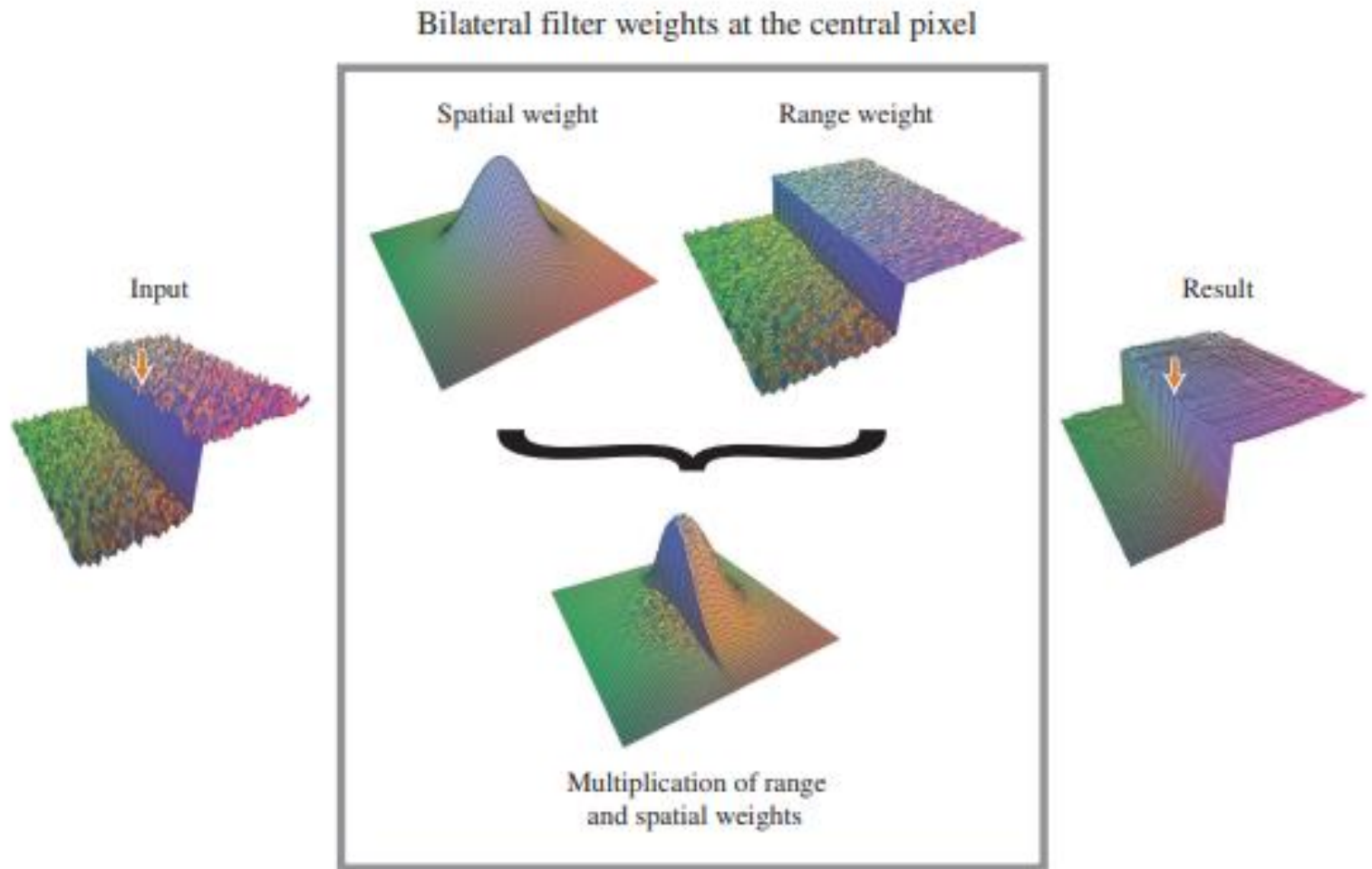


Figure . The bilateral filter smooths an input image while preserving its edges.[1]

Chapter 3. Image processing

References

[1] Sylvain Paris, Pierre Kornprobst, Jack Tumblin and Frédo Durand.
Bilateral Filtering: Theory and Applications. Foundations and Trends in
Computer Graphics and Vision Vol. 4, No. 1 (2008) 1–73